# Seminario de Arquitecturas de Software:
# Un Análisis Comparativo de Estilos Basados en Capas y Dominio

Ingeniería de Software

**Equipo:** CELL

**Integrantes y preguntas asignadas**

| **Integrante** | **Pregunta** | **Tema** |
|---|---|---|
| Eduard Gonzáles Prieto | 1 | Análisis de *trade-offs*: N-Capas frente a Clean/Onion/Hexagonal |
| Manuel Alejandro Rodríguez | 2 | La simetría del hexágono frente a la jerarquía de las capas |
| Reynol Gómez Franco | 3 | DDD y *Bounded Context* en un dominio complejo |
| Luis David Díaz | 4 | Inversión de dependencias en la práctica |
| Cristian Miguel Castañeda | 5 | El *Kitty Problem* y el Principio de Abierto/Cerrado |
| Lázaro Javier Aragon | 6 | N-Capas frente a requisitos de atributos de calidad |

**Repositorio público:**

https://github.com/ReynolGF/repo_IS_BD2_equipo_CELL

23 de octubre de 2026

# Índice

**1. Análisis de *trade-offs*: Arquitectura N-Capas frente a Clean/Onion/Hexagonal** **2**

**2. Comparación de estilos: la simetría del hexágono frente a la jerarquía de las capas** **3**

**3. Aplicación de DDD y *Bounded Context* en un dominio complejo** **9**

**4. Evaluación de la inversión de dependencias en la práctica** **13**

**5. Análisis de un problema real: el *Kitty Problem* y el Principio de Abierto/Cerrado** **21**

**6. Evaluación de la Arquitectura N-Capas frente a requisitos de atributos de calidad** **23**

**Referencias** **25**

**Declaración de Uso de la Inteligencia Artificial** **25**

# 1. Análisis de *trade-offs*: Arquitectura N-Capas frente a Clean/Onion/Hexagonal

*Responsable: Eduard Gonzáles Prieto*

### Identificación del error de acoplamiento

El equipo de desarrollo ha caído en el error fundamental de acoplamiento que caracteriza a las implementaciones rígidas de la Arquitectura N-Capas tradicional: la dependencia del flujo de control y de las capas externas hacia las internas. En este diseño, la capa de negocio (el núcleo) depende directamente de la capa de datos (base de datos SQL Server) y de la capa de presentación. Esto viola el principio de inversión de dependencias, ya que el núcleo de negocio, que contiene las reglas más valiosas y estables del sistema, está acoplado a los detalles volátiles (tecnología de BD, interfaz de usuario), lo que convierte al sistema en un «monolito» difícil de mantener y probar (Martin, 2017; Richards, 2015).

### La solución estructural: Regla de Dependencia y Puertos/Adaptadores

Para resolver esto, la Arquitectura Limpia (Martin, 2017) establece la Regla de Dependencia: las dependencias del código fuente solo pueden apuntar hacia adentro, hacia las capas de mayor nivel (Entidades y Casos de Uso). La lógica de negocio no debe saber nada de la base de datos ni de la UI. Por su parte, la Arquitectura Hexagonal (Cockburn, 2005) propone el uso de Puertos y Adaptadores. El núcleo de la aplicación define «puertos» (interfaces) que expresan lo que necesita del exterior (por ejemplo, un puerto para guardar un producto). Los «adaptadores» (implementaciones concretas, como el repositorio de SQL Server o MongoDB) se conectan a esos puertos desde el exterior. Así, el flujo de dependencias se invierte: la infraestructura depende de la abstracción del dominio, y no al revés.

### Impacto en la testabilidad y el cambio de base de datos

El impacto es transformador. Al aplicar esta inversión de dependencias, la lógica de negocio (el núcleo) queda completamente aislada.

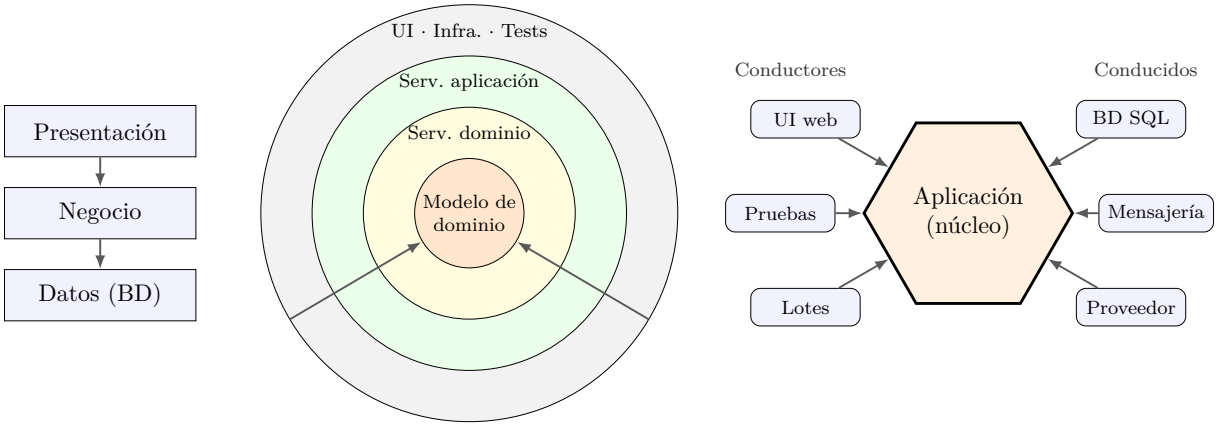
- **Testabilidad:** las pruebas unitarias de la lógica de negocio ya no necesitan una base de datos real ni una interfaz de usuario. Se pueden crear *mocks* o simulaciones de los adaptadores, lo que hace que las pruebas sean rapidísimas, robustas y altamente confiables (Martin, 2017).
- **Cambio de base de datos (SQL Server a MongoDB):** para migrar de base de datos, el equipo solo tendría que crear un nuevo adaptador (por ejemplo, `MongoDBRepository`) que implemente la misma interfaz (el puerto) que ya definió el dominio. La lógica de negocio no se tocaría ni una sola línea, ya que sigue apuntando a la abstracción (el puerto) y no a la implementación concreta. Esto cumple con el principio de que «los detalles (la BD) deben depender de las abstracciones (las políticas de negocio)» (Martin, 2017).

## 2. Comparación de estilos: la simetría del hexágono frente a la jerarquía de las capas

*Responsable: Manuel Alejandro Rodríguez*

### Planteamiento

La Arquitectura Hexagonal de Cockburn (2005) y la Arquitectura Cebolla de Palermo (2008a) persiguen el mismo fin: que el núcleo de negocio no dependa de la tecnología que lo rodea. Ambas hacen que las dependencias apunten hacia el núcleo y ambas tratan la interfaz de usuario y la base de datos como detalles reemplazables. Sin embargo, eligen metáforas visuales distintas, y esa elección no es decorativa: un diagrama de arquitectura condiciona qué preguntas se hace el equipo, qué decisiones parecen naturales y qué errores pasan inadvertidos. A continuación se analiza (1) qué problema conceptual resuelve la simetría del hexágono, (2) cómo se traducen los puertos y adaptadores en interfaces de las capas internas de la Cebolla y (3) por qué el número seis es irrelevante.




**(a) N-Capas** **(b) Cebolla** **(c) Hexagonal**

Figura 1: Tres metáforas visuales para un mismo objetivo. Las flechas representan dependencias del código fuente: en (a) descienden hacia los datos; en (b) apuntan al centro, pero el dibujo enfatiza la profundidad de los anillos; en (c) todo actor externo se conecta mediante un puerto y todas las dependencias apuntan al interior. Elaboración propia a partir de Richards (2015), Palermo (2008a) y Cockburn (2005).

### 1. El problema conceptual que resuelve la simetría

**Lo que comunica una pila de capas.** En la arquitectura N-Capas tradicional el diagrama es unidimensional y vertical: presentación arriba, negocio en medio y datos abajo, con dependencias que descienden (Richards, 2015). Sin decirlo explícitamente, esta imagen transmite tres ideas que conducen a malas decisiones de diseño:

1. **Una jerarquía de cimientos.** Lo de abajo «sostiene» a lo de arriba, de modo que la base de datos se percibe como el fundamento del sistema y el diseño suele comenzar por el esquema de datos. Palermo (2008a) señala que el gran inconveniente de esta arquitectura es el acoplamiento que genera: cada capa queda atada a las inferiores y, a menudo, a la infraestructura. Su respuesta es tajante: la base de datos no es el centro, sino algo externo.

2. **Dos extremos de naturaleza distinta.** La interfaz de usuario «arriba» y la base de datos «abajo» parecen problemas diferentes que merecen tratamientos diferentes. Esto abre dos vías

de fuga para la lógica de negocio: hacia arriba, en controladores y formularios —Cockburn (2005) presenta la infiltración de la lógica de negocio en el código de la interfaz de usuario como uno de los males históricos que motivan su propuesta—, y hacia abajo, en consultas y procedimientos almacenados.

3. **Solo hay dos lados.** Una pila tiene un extremo superior y uno inferior. No ofrece un lugar evidente para una batería de pruebas automatizadas, un proceso por lotes, otra aplicación que invoca por API, una cola de mensajes o un proveedor externo; esos actores acaban «colgados» de alguna capa o forzados a entrar por la interfaz gráfica.

**Lo que comunica el hexágono.** Cockburn sustituye el eje arriba/abajo por la única distinción que considera esencial: **dentro/fuera**. Para él, los problemas del lado del usuario y los del lado del servidor nacen, en realidad, del mismo error de diseño; por eso la asimetría que conviene explotar no es la que hay entre los lados izquierdo y derecho de la aplicación, sino entre su interior y su exterior, y la regla que debe cumplirse es que el código del interior no se filtre hacia el exterior (Cockburn, 2005). Dentro está la aplicación (reglas de negocio y casos de uso); fuera, todo lo demás. Desde el punto de vista del núcleo, un usuario humano, una batería de pruebas, un script por lotes, una base de datos o un servicio de terceros son lo mismo: actores externos que conversan con la aplicación a través de un *puerto*, mediante un *adaptador* propio de cada tecnología. El objetivo declarado es que la aplicación pueda ser conducida por igual por usuarios, programas, pruebas automatizadas o scripts por lotes, y que se desarrolle y pruebe aislada de sus dispositivos y bases de datos de ejecución (Cockburn, 2005).

La simetría resuelve así varios problemas que la pila no logra transmitir de forma intuitiva:

- **Trato uniforme del exterior.** La interfaz de usuario y la base de datos dejan de ser «la capa superior» y «la capa inferior» para convertirse en adaptadores del mismo tipo, sustituibles con el mismo mecanismo. Martin (2017) llega a la misma conclusión desde otra perspectiva: la interfaz de usuario y la base de datos son *detalles*, *plugins* del núcleo de negocio.
- **Una frontera visible.** El hexágono dibuja un contorno cerrado; cruzarlo es un acto explícito que exige un puerto. En una pila, la frontera entre negocio y datos es una línea más entre capas y nada sugiere que cruzarla deba ser una decisión de diseño.
- **Las pruebas como actor de primera clase.** Una batería de pruebas es un adaptador más, situado en el mismo borde que la interfaz de usuario. La testabilidad deja de ser un efecto secundario y pasa a ser una consecuencia directa de la forma.
- **Pluralidad.** El dibujo admite tantos puertos como conversaciones tenga la aplicación y varios adaptadores por puerto (por ejemplo, un repositorio SQL en producción y otro en memoria para las pruebas), algo que una pila de tres cajas no sugiere.

La simetría del dibujo no implica que todos los actores desempeñen el mismo papel. Cockburn (2005) advierte que el patrón está escrito, deliberadamente, como si todos los puertos fueran fundamentalmente iguales —una ficción útil en el nivel arquitectónico—, pero que en la implementación aparecen dos variantes: los puertos y adaptadores *primarios* (o conductores), ligados a actores que conducen la aplicación, y los *secundarios* (o conducidos), ligados a actores a los que la aplicación conduce para obtener respuestas o simplemente para notificarles algo. Por eso propone dibujar los primarios en el lado izquierdo (o arriba) del hexágono y los secundarios en el derecho (o abajo). Pero se trata de una diferencia de *rol en la conversación* —quién la inicia—, no de rango ni de dirección de las dependencias: ambos lados siguen el mismo patrón de puerto y adaptador, y ninguno está «por encima» del núcleo.

**¿Y la Cebolla?** El enunciado agrupa la Cebolla con las representaciones de capas, y conviene matizarlo. Los anillos concéntricos de Palermo (2008a) ya corrigen buena parte del problema de la pila: la base de datos sale del fondo y pasa al anillo exterior; la interfaz de usuario, la infraestructura y las pruebas comparten ese mismo anillo, lo que ya es una forma de simetría; y todas las dependencias apuntan hacia el centro. Lo que conserva es una metáfora de *profundidad*: Modelo de Dominio, Servicios de Dominio, Servicios de Aplicación y, por último, el exterior. El dibujo responde bien a la pregunta «¿qué anillo envuelve a cuál?», pero no muestra a través de qué *conversaciones* entra cada actor externo ni que un mismo contrato admite varias implementaciones: el anillo exterior aparece como una corteza homogénea. Además, cuando la Cebolla se traslada a la estructura de una solución (proyectos `Core`, `Infrastructure` y `UI` listados uno debajo de otro), la lectura vertical tiende a reaparecer. En síntesis, la Cebolla explica cómo organizar el interior en profundidad; el hexágono explica cómo se relaciona ese interior con un exterior plural. Son representaciones complementarias (Tabla 1).

| **Aspecto** | **N-Capas (pila)** | **Cebolla (anillos)** | **Hexagonal (hexágono)** |
|---|---|---|---|
| Eje que transmite | Vertical: arriba/abajo | Radial, con niveles de profundidad | Dentro/fuera |
| Lugar de la base de datos | El fondo: el «cimiento» del sistema | Anillo exterior (Infraestructura) | Un adaptador conducido más |
| Lugar de la UI y de las pruebas | UI en la cima; las pruebas, sin lugar propio | Anillo exterior, junto a la infraestructura | Adaptadores conductores en el mismo borde |
| Dirección de las dependencias | Hacia abajo, hacia los datos | Hacia el centro | Hacia el interior, a través de puertos |
| Qué hace explícito | La separación técnica por roles | La organización interna del núcleo | Las conversaciones (puertos) y su multiplicidad |
| Lectura errónea que induce | Diseño centrado en los datos; fugas de lógica hacia la UI y la BD | Exterior percibido como una corteza homogénea | Pasar por alto la organización interna del núcleo |

Tabla 1: Qué comunica cada representación visual. Elaboración propia a partir de Richards (2015), Palermo (2008a) y Cockburn (2005).

## 2. Del Puerto y el Adaptador a las interfaces de las capas internas de la Cebolla

Palermo (2008b) enuncia cuatro principios: la aplicación se construye alrededor de un modelo de objetos independiente; las capas internas definen interfaces y las externas las implementan; el acoplamiento se dirige hacia el centro; y todo el código del núcleo puede compilarse y ejecutarse separado de la infraestructura. El segundo principio es, literalmente, el patrón de puertos y adaptadores expresado en términos de capas: **el puerto es la interfaz que declara una capa interna y el adaptador es la clase de un anillo exterior que la implementa**. Palermo (2008a) lo ilustra con las interfaces de repositorio: residen en la primera capa alrededor del Modelo de Dominio, mientras que su implementación, que involucra una base de datos, queda en el borde; en el núcleo solo está la interfaz.

La traducción práctica distingue los dos tipos de puerto y el mecanismo que los une:

- **Puertos conducidos (secundarios):** lo que el núcleo necesita del exterior (persistir pedidos, notificar a un cliente, cobrar un pago). Se expresan como interfaces declaradas en los anillos internos (`IPedidoRepository`, `INotificadorCliente`) y se implementan en el anillo

de Infraestructura mediante adaptadores (`SqlPedidoRepository`, `MongoPedidoRepository`, `SmtpNotificadorCliente`).

- **Puertos conductores (primarios):** lo que el exterior puede pedir al núcleo. En la Cebolla corresponden a los Servicios de Aplicación, es decir, a los casos de uso (`RegistrarPedidoService`). Los adaptadores conductores —un controlador REST, un consumidor de mensajes, un comando de consola o una prueba automatizada— viven en el anillo exterior (UI y Tests) y solo conocen esa API.
- **El ensamblaje:** un mecanismo de inyección de dependencias en tiempo de ejecución conecta cada interfaz con su implementación (Palermo, 2008a). Equivale al componente *Main* de Martin (2017) y al contenedor IoC de la guía de De la Torre Llorente et al. (2010): es el único lugar que nombra clases concretas.

El Código 1 muestra esta correspondencia en C#. Las referencias entre proyectos son `Infrastructure` → `Core` y `Api` → `Core`; `Core` no referencia a ningún otro proyecto.

Código 1: Puertos declarados en los anillos internos de la Cebolla y adaptadores en el anillo exterior.

```
// Pedidos.Core, anillo de Servicios de Dominio: PUERTO conducido
public interface IPedidoRepository
{
    Pedido? ObtenerPorId(PedidoId id);
    void Guardar(Pedido pedido);
}

// Pedidos.Core, anillo de Servicios de Aplicación: PUERTO conductor (caso de uso)
public class RegistrarPedidoService
{
    private readonly IPedidoRepository _pedidos;            // solo conoce la abstracción
    public RegistrarPedidoService(IPedidoRepository pedidos) => _pedidos = pedidos;

    public PedidoId Ejecutar(RegistrarPedidoCommand cmd)
    {
        var pedido = Pedido.Crear(cmd.ClienteId, cmd.Lineas); // regla de negocio
        _pedidos.Guardar(pedido);
        return pedido.Id;
    }
}

// Pedidos.Infrastructure, anillo exterior: ADAPTADOR conducido
public class SqlPedidoRepository : IPedidoRepository
{
    private readonly PedidosDbContext _db;                  // la tecnología queda aquí
    public SqlPedidoRepository(PedidosDbContext db) => _db = db;

    public Pedido? ObtenerPorId(PedidoId id) => _db.Pedidos.Find(id);
    public void Guardar(Pedido pedido) { _db.Pedidos.Add(pedido); _db.SaveChanges(); }
}

// Pedidos.Api, anillo exterior: ADAPTADOR conductor
[ApiController, Route("pedidos")]
public class PedidosController : ControllerBase
{
    private readonly RegistrarPedidoService _registrar;
    public PedidosController(RegistrarPedidoService registrar) => _registrar = registrar;

    [HttpPost]
    public IActionResult Post(RegistrarPedidoCommand cmd) => Ok(_registrar.Ejecutar(cmd));
}
```

```
// Program.cs, composición: único lugar que nombra las clases concretas
builder.Services.AddScoped<IPedidoRepository, SqlPedidoRepository>();
builder.Services.AddScoped<RegistrarPedidoService>();

// Pedidos.Tests: otro adaptador conductor, con un adaptador conducido en memoria
var servicio = new RegistrarPedidoService(new PedidoRepositoryEnMemoria());
```

Migrar la persistencia a MongoDB consiste en escribir `MongoPedidoRepository : IPedidoRepository` y cambiar una línea del punto de composición: ni `RegistrarPedidoService` ni sus pruebas se modifican. La Tabla 2 resume la equivalencia entre ambos vocabularios.

| **Hexagonal (Cockburn, 2005)** | **Cebolla (Palermo, 2008a)** | **Ejemplo** |
|---|---|---|
| Interior del hexágono | Modelo de Dominio, Servicios de Dominio y Servicios de Aplicación | `Pedido`, `RegistrarPedidoService` |
| Puerto conductor (primario) | API de los Servicios de Aplicación | `RegistrarPedidoService` |
| Adaptador conductor | Anillo exterior: UI y Tests | `PedidosController`, pruebas |
| Puerto conducido (secundario) | Interfaz definida en una capa interna | `IPedidoRepository` |
| Adaptador conducido | Anillo exterior: Infraestructura | `SqlPedidoRepository`, `MongoPedidoRepository` |
| Configuración de los adaptadores | Inyección de dependencias en tiempo de ejecución | `Program.cs` |

Tabla 2: Correspondencia entre el vocabulario hexagonal y los anillos de la Cebolla.

Para que una interfaz de la Cebolla sea realmente un puerto, y no solo una interfaz más, deben cumplirse cuatro criterios:

1. **Propiedad.** La interfaz se declara en el proyecto del núcleo, junto a quien la usa, y no junto a quien la implementa. Si `IPedidoRepository` viviera en Infraestructura, el núcleo tendría que referenciar ese proyecto y la dependencia volvería a apuntar hacia fuera; es lo que exige el Principio de Inversión de Dependencias (Martin, 2017).

2. **Lenguaje del dominio.** La firma usa tipos del dominio (`Pedido`, `PedidoId`) y nunca tipos tecnológicos (`DbContext`, `DataRow`, `HttpResponseMessage`, `BsonDocument`). Una interfaz que expone la tecnología es un puerto con fugas.

3. **Una conversación con propósito.** Cockburn (2005) define el puerto por el propósito de la conversación, no por la tecnología; por eso se nombra por su capacidad (`INotificadorCliente`) y no por su mecanismo (`ISmtpGateway`).

4. **Ubicación en profundidad.** Aquí la Cebolla aporta algo que el hexágono deja abierto: en qué anillo vive cada puerto. Las interfaces de repositorio se sitúan junto al Modelo de Dominio, mientras que las de servicios externos que solo sirven a un caso de uso suelen pertenecer a los Servicios de Aplicación. La guía de De la Torre Llorente et al. (2010) sigue el mismo esquema: los contratos de los repositorios se definen en la capa de Dominio y sus implementaciones, en la capa de Infraestructura de Persistencia.

Con ello, la diferencia entre ambos estilos se reduce a una cuestión de énfasis: la Arquitectura Hexagonal describe la frontera y no prescribe cómo se organiza el interior; la Cebolla añade una

estratificación interna del núcleo. Vista desde fuera, una aplicación Cebolla bien construida es un hexágono.

## 3. Por qué el hexágono no tiene seis lados por una razón numérica

Cockburn (2005) aclara que la figura no es un hexágono porque el número seis sea importante, sino para dejar a quien dibuja espacio suficiente para insertar los puertos y adaptadores que necesite, sin las restricciones de un dibujo de capas unidimensional; el nombre «hexagonal» proviene únicamente de ese efecto visual. Según el mismo autor, la forma pretende resaltar dos cosas: la asimetría dentro/fuera y la naturaleza similar de los puertos, para alejarse de la imagen unidimensional de capas y de todo lo que evoca, y la existencia de un número definido de puertos distintos. Ese número no guarda relación con los lados de la figura: Cockburn reconoce que decidir qué es exactamente un puerto es, en gran medida, cuestión de gusto, que elegir un número «equivocado» no parece causar ningún daño y que su preferencia se inclina por pocos puertos —dos, tres o cuatro—; además, cada puerto suele tener varios adaptadores, uno por tecnología. De ahí que el patrón se conozca también como «Puertos y Adaptadores», un nombre que describe mejor sus piezas que la forma del dibujo.

El argumento es geométrico:

- **Una pila es un segmento.** Tiene exactamente dos extremos, impone un «arriba» y un «abajo» y sugiere que hay justo dos tipos de exterior, de rango distinto. Girar el dibujo noventa grados y colocar la interfaz a la izquierda y la base de datos a la derecha no corrige nada: la asimetría solo se traslada al eje horizontal.
- **Un polígono regular no tiene orientación privilegiada.** Ninguna cara es «la de arriba», cualquiera puede alojar cualquier puerto y, si se gira una sexta parte de vuelta, la figura es la misma. Esa invariancia comunica que todos los actores externos son equivalentes frente al núcleo y que la única asimetría con significado arquitectónico es la de dentro/fuera.
- **Un compromiso práctico.** Como interpretación propia, el hexágono tiene caras suficientes para repartir varios puertos alrededor del núcleo y, a la vez, es fácil de dibujar a mano en una pizarra. Un círculo también sería simétrico, pero no ofrece caras en las que «enchufar» cada conversación; un triángulo tiene muy pocos lados y un cuadrado se lee con facilidad como una caja con arriba y abajo.

Así, el hexágono elimina las asimetrías que no importan (arriba-abajo e izquierda-derecha entendidas como rango o como capas) y preserva la única que sí importa: la que separa la aplicación de su entorno. La convención que el propio Cockburn propone —primarios a la izquierda (o arriba), secundarios a la derecha (o abajo)— no contradice esta idea, porque no expresa dependencias ni jerarquía, sino quién inicia la conversación.

## 4. Conclusión

Hexagonal y Cebolla protegen el mismo núcleo con el mismo mecanismo —interfaces definidas dentro, implementaciones fuera y dependencias hacia el centro—, pero enseñan cosas distintas. La pila de la N-Capas induce a pensar en jerarquías y en la base de datos como cimiento; los anillos de la Cebolla corrigen la dirección de las dependencias y explican la organización interna del núcleo; el hexágono hace visible la frontera y la pluralidad de actores externos, y trata a la interfaz, la base de datos y las pruebas como lo que son ante el núcleo: adaptadores intercambiables conectados a puertos. Que la figura tenga seis lados es irrelevante; lo relevante es que no tiene ni arriba ni abajo.

# 3. Aplicación de DDD y *Bounded Context* en un dominio complejo

*Responsable: Reynol Gómez Franco*

## Planteamiento

Una aseguradora desea construir un sistema para gestionar pólizas de automóviles que interactúe con tres subsistemas con lenguajes y preocupaciones propias: el departamento de siniestros, el sistema de facturación de primas y la aplicación móvil de los ajustadores que evalúan daños en el lugar del accidente. El término «póliza» no significa lo mismo en cada uno de ellos: para un actuario es un contrato con cláusulas y coberturas, para facturación es una fuente de ingresos y de cuotas, y para un ajustador es apenas un número de referencia que identifica el siniestro que debe inspeccionar.

Intentar un único modelo de `Poliza` que satisfaga a todos esos actores conduce al problema clásico que Evans (2003) denomina *modelo único para todo el negocio*: un modelo sobrecargado, con campos y comportamientos que solo interesan a un área, acoplado a bases de datos compartidas y cada vez más difícil de modificar. La respuesta de *Domain-Driven Design* (DDD) no es construir un modelo más grande, sino dividir el dominio en varios *contextos delimitados* (*Bounded Contexts*), cada uno con su propio *lenguaje ubicuo* y su propio modelo, y definir entre ellos relaciones explícitas y contratos de integración (Evans, 2003; Richards & Ford, 2020).

## 1. División del sistema en contextos delimitados

La primera decisión de diseño es delimitar las fronteras del modelo, no las de la organización ni las de la interfaz de usuario. Cada contexto debe agrupar un conjunto coherente de conceptos y reglas que un mismo equipo puede mantener y que usan un mismo lenguaje. Para este sistema se proponen los siguientes contextos:

La delimitación sigue el criterio de Evans (2003): una frontera de contexto es una frontera de modelo y de lenguaje. Nótese que «Póliza» aparece en varios contextos, pero con significados y ciclos de vida diferentes; por eso existen varias clases `Poliza`, una por contexto, y no una clase compartida. Esto es intencional: compartir el modelo sería precisamente la fuente de contaminación que se quiere evitar.

## 2. Cómo evitar que el modelo de «Póliza» se contamine

La contaminación se evita con cuatro mecanismos complementarios.

1. **Un modelo por contexto y propiedad exclusiva de los datos.** Cada contexto posee su esquema de base de datos y solo ese contexto escribe en él. Siniestros y Facturación no acceden a las tablas de Suscripción; obtienen la información por un contrato de integración. No existen *joins* ni claves foráneas entre contextos, ni entidades JPA/EF compartidas entre módulos.

2. **Referencias por identidad, no por objeto.** En Siniestros, la póliza se representa mediante un objeto de valor `PolicyId` y, cuando se requiere el detalle, mediante una *fotografía* (*snapshot*) inmutable de las cláusulas y coberturas vigentes en la fecha del siniestro. Así, una modificación posterior de la póliza (un endoso) no altera retroactivamente un siniestro ya dictaminado, y el modelo de Siniestros no necesita conocer la estructura interna del contrato.

3. **Capa anticorrupción (*Anti-Corruption Layer*, ACL).** Todo dato que provenga de otro contexto pasa por un traductor que convierte el modelo externo en el modelo local. La ACL protege el lenguaje ubicuo: si el sistema de facturación cambia sus campos o su semántica,

| Contexto | Lenguaje ubicuo | Responsabilidad / modelo | Tipo (DDD) |
|---|---|---|---|
| **Suscripción y Pólizas** | Póliza, cláusula, cobertura, vigencia, endoso, asegurado, vehículo | Contrato de seguro: emisión, modificaciones, cancelación, reglas de cobertura y versión del contrato. Es la fuente de verdad del contrato. | Núcleo (*core domain*) |
| **Siniestros** | Siniestro, reclamación, ocurrencia, beneficiario, dictamen, cobertura aplicable | Registro y tramitación del siniestro; determina qué coberturas aplican según la póliza vigente al momento del hecho. | Núcleo |
| **Facturación de Primas** | Prima, cuota, recibo, mora, nota de crédito, conciliación | Generación y cobro de las obligaciones económicas derivadas de la póliza; no interpreta cláusulas, solo montos y vencimientos. | Soporte |
| **Ajustes en Campo** | Ajuste, inspección, daño, evidencia, peritaje, número de referencia | Asignación, ejecución y cierre de inspecciones de daños; trabaja con datos mínimos del siniestro y del vehículo. | Soporte |
| **Catálogo y Tarifas** | Producto, plan, tarifa, factor de riesgo, cobertura comercial | Definición de productos y precios que consumen Suscripción y Facturación. | Genérico / soporte |
| **Identidad y Notificaciones** | Usuario, ajustador, asegurado, canal, plantilla | Autenticación, autorización y envío de comunicaciones. | Genérico |

Tabla 3: Contextos delimitados propuestos y su lenguaje ubicuo.

solo se modifica el traductor, no el dominio de Siniestros (Evans, 2003; Richards & Ford, 2020).

4. **Fronteras verificadas automáticamente.** Las dependencias entre contextos se restringen por paquetes/*namespaces* y se validan en integración continua con pruebas de arquitectura (por ejemplo, ArchUnit en la JVM o reglas equivalentes en .NET). Un contexto no puede importar clases de dominio de otro; solo puede usar sus contratos de integración.

En consecuencia, el modelo de Póliza del contexto de Suscripción conserva únicamente conceptos contractuales (cláusulas, coberturas, vigencias, endosos). Facturación modela una `CuentaDeCobro` con cuotas y recibos derivados de eventos, y Ajustes modela una `AsignacionDeAjuste` con un número de referencia, sin heredar ni replicar el contrato completo.

## 3. Comunicación desacoplada entre contextos

Los contextos se integran mediante **eventos de dominio publicados como eventos de integración** y mediante **contratos explícitos** (interfaces de consulta o APIs), nunca mediante una base de datos compartida. Los patrones de *context mapping* de Evans (2003) permiten nombrar cada relación:

- **Suscripción como *Open Host Service* con *Published Language***: publica un contrato estable (por ejemplo, un esquema de eventos versionado) que otros contextos consumen sin conocer su modelo interno.
- **Siniestros y Facturación como consumidores con ACL**: cada uno traduce esos eventos a su propio modelo. La relación es de tipo *customer–supplier*: el consumidor define sus necesidades y el proveedor se compromete a mantener el contrato.

- **Catálogo y Tarifas como proveedor conforme** (*conformist*) o con ACL, según la rigidez del contrato: si el catálogo es corporativo e inmutable, los demás se adaptan a él; si no, se interpone una ACL.
- **Identidad y Notificaciones como servicio genérico** consumido por todos, con relaciones de tipo *separate ways* respecto del núcleo.

Los eventos típicos son `PolicyIssued`, `PolicyEndorsed`, `PolicyCancelled`, `ClaimRegistered`, `ClaimApproved` y `PaymentReceived`. La comunicación es **asíncrona** a través de un bus de mensajes, lo que desacopla en el tiempo y en el espacio a emisores y consumidores y evita que una caída de Facturación detenga la atención de un siniestro. Para no perder eventos se aplica el patrón *outbox* transaccional, y para no duplicar efectos, consumidores idempotentes. Los procesos que abarcan varios contextos (por ejemplo, aprobar un siniestro y generar la indemnización en Facturación) se coordinan con una *saga* o *process manager*, aceptando consistencia eventual (Richards & Ford, 2020).

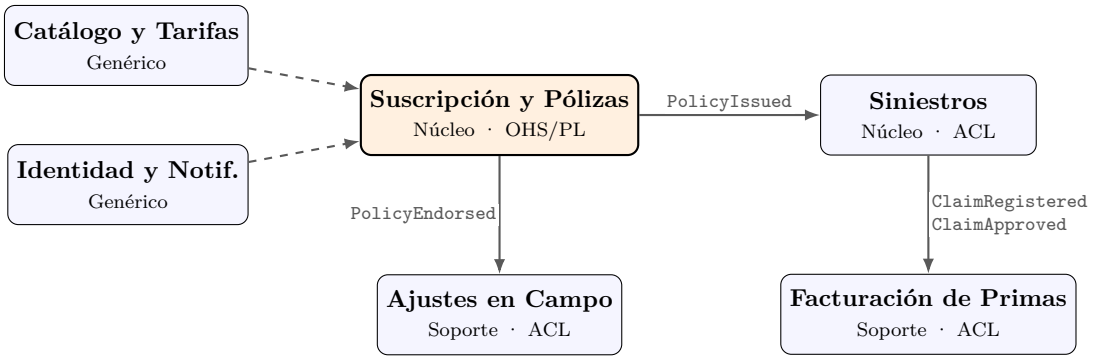



Figura 2: Mapa de contextos: contratos de integración y eventos principales.

## 4. Relación con una arquitectura orientada al dominio (Onion/Clean)

El diseño anterior es independiente del estilo de despliegue, pero encaja naturalmente con la Arquitectura Limpia y la Arquitectura Cebolla: **cada contexto delimitado se implementa como un núcleo de dominio aislado**, y las integraciones son detalles reemplazables (Martin, 2017; Palermo, 2008a).

- **Centro (dominio):** entidades, objetos de valor, agregados, eventos de dominio y reglas invariantes del contexto. No conoce frameworks, bases de datos ni HTTP.
- **Aplicación:** casos de uso y *puertos* de entrada y de salida. La Regla de Dependencia garantiza que el código fuente solo apunte hacia el centro.
- **Infraestructura:** adaptadores que implementan los puertos (repositorios, mensajería, clientes HTTP). Aquí viven las ACL y los traductores entre modelos.
- **Interfaces:** controladores REST, consumidores de eventos y la aplicación móvil del ajustador, como adaptadores de entrada.

La «Guía de Arquitectura N-Capas orientada al Dominio» de De la Torre Llorente et al. (2010) describe este mismo mecanismo con el nombre de **Adaptador de Servicio**: un componente que traduce el contrato de un servicio externo al contrato que espera la capa de aplicación, aislando al dominio de la tecnología del proveedor. En términos de Arquitectura Limpia, el Adaptador de Servicio es un *adaptador de salida* que implementa un *puerto* definido en el núcleo. Por ejemplo, cuando Siniestros necesita conocer la póliza, el dominio define el puerto y la infraestructura aporta el adaptador:

Código 2: Puerto en el núcleo y Adaptador de Servicio en infraestructura.

```
// Nucleo (capa de dominio/aplicacion): el puerto pertenece al contexto
public interface PolicyQueryPort {
    PolicySnapshot findSnapshotBy(PolicyId id, LocalDate occurrenceDate);
}

// Infraestructura: Adaptador de Servicio + Anti-Corruption Layer
public final class PolicyServiceAdapter implements PolicyQueryPort {
    private final PolicyApiClient client;      // detalle tecnologico
    private final PolicyTranslator translator; // ACL

    @Override
    public PolicySnapshot findSnapshotBy(
            PolicyId id, LocalDate date) {
        PolicyDto dto = client.getPolicy(id.value(), date);
        return translator.toSnapshot(dto); // sin PolicyDto en el dominio
    }
}
```

El dominio de Siniestros depende de `PolicyQueryPort`, no de `PolicyApiClient`; la inyección de dependencias se resuelve en el *composition root* de infraestructura. El mismo esquema aplica a la publicación y consumo de eventos: el dominio declara `DomainEventPublisher` y la infraestructura lo implementa sobre el bus de mensajes. Así, la Arquitectura Orientada al Dominio no solo organiza las capas de cada contexto, sino que hace explícitos los límites entre contextos: cada frontera coincide con un puerto y con un contrato versionado.

## 5. Consecuencias y conclusión

La división propuesta produce un sistema en el que (i) cada área mantiene su propio lenguaje y su propio ritmo de cambio; (ii) el modelo de Póliza del núcleo permanece protegido por identidades, fotografías y ACL; (iii) los contextos se comunican por eventos y contratos versionados, sin bases de datos compartidas; y (iv) cada contexto puede implementarse como un núcleo Onion/Clean con adaptadores de servicio intercambiables. El costo es asumir consistencia eventual y una mayor complejidad operativa, que se justifica en un dominio complejo donde la autonomía de los equipos y la evolucionabilidad del negocio son requisitos de primer orden (Evans, 2003; Richards & Ford, 2020).

# 4. Evaluación de la inversión de dependencias en la práctica

*Responsable: Luis David Díaz*

### Planteamiento y respuesta resumida

El enunciado sitúa en `OrderProcessor` la lógica de negocio principal del sistema de pedidos (qué hacer con un pedido cuya dirección es válida, inválida o no verificable), por lo que pertenece al núcleo. Esta clase instancia directamente a `AddressValidatorService`, que usa una librería HTTP concreta para consultar un servicio externo de validación de direcciones. Los ejemplos usan C#, pero el análisis no depende del lenguaje.

**Tesis.** El problema no es consumir un servicio externo ni usar HTTP, sino la **dirección** de la dependencia: una política de alto nivel depende de un mecanismo de bajo nivel. Se corrige invirtiéndola en la frontera del núcleo.

- **a)** La cadena `OrderProcessor` → `AddressValidatorService` → librería HTTP apunta del núcleo hacia afuera y viola el DIP, la Regla de Dependencia, el aislamiento por puertos y los principios de la Cebolla. Las dependencias del código fuente deben apuntar siempre hacia el núcleo.
- **b)** El núcleo declara y posee el puerto `IAddressValidator`, en lenguaje del dominio. La infraestructura lo implementa con un adaptador que encapsula la librería HTTP, y un punto de composición los conecta por inyección de dependencias.
- **c)** `OrderProcessor` se prueba en memoria con un doble de prueba, sin red ni librería HTTP. Cambiar de proveedor, de librería o de protocolo se reduce a tocar un adaptador.

### Escenario de partida

```
// ANTES: el núcleo depende de un detalle concreto y volátil
public class OrderProcessor
{
    public async Task ProcessAsync(Order order)
    {
        var validator = new AddressValidatorService();              // (1)
        if (!await validator.IsValidAsync(order.ShippingAddress))   // (2)
            throw new InvalidOperationException("Dirección inválida");
        // ... resto de las reglas del pedido
    }
}

public class AddressValidatorService
{
    public async Task<bool> IsValidAsync(Address address)
    {
        using var http = new HttpClient();                          // (3)
        var response = await http.PostAsJsonAsync(
            "https://api.proveedor.com/v1/validate", address);
        return response.IsSuccessStatusCode;                        // (4)
    }
}
```

Hay dos acoplamientos distintos, y cada uno exige su propia solución:

- **Acoplamiento a la implementación (1, 2):** `OrderProcessor` crea y nombra una clase concreta.

- **Acoplamiento a la tecnología (3, 4):** la clase concreta conoce la librería HTTP y devuelve semántica HTTP en lugar de semántica del dominio (un 2xx indica que la petición tuvo éxito, no que la dirección sea válida). Como las dependencias son transitivas, `OrderProcessor` hereda también las razones de cambio de la librería y del proveedor.

## a) Por qué la dependencia directa viola los principios de las arquitecturas Hexagonal, Limpia y Cebolla

### 1. Dirección actual de las dependencias

```
OrderProcessor            (política de negocio)
      |  depende de
      v
AddressValidatorService   (detalle)
      |  depende de
      v
Librería HTTP             (detalle)
      |
      v
Proveedor externo         (exterior)
```

El flujo de control y las dependencias del código fuente coinciden y apuntan hacia afuera. Es el mismo acoplamiento de arriba hacia abajo que Palermo (2008a) critica en las capas tradicionales, donde cada capa queda atada a las inferiores y, al final, a la infraestructura.

### 2. Qué principio se viola en cada estilo

- **Inversión de Dependencias (DIP).** Los sistemas más flexibles son aquellos cuyo código fuente se refiere solo a abstracciones y no a concreciones (Martin, 2017, cap. 11). En su formulación clásica: los módulos de alto nivel no deben depender de los de bajo nivel (ambos dependen de abstracciones) y los detalles deben depender de las abstracciones, no al revés. Martin lo resume en no mencionar nunca el nombre de nada concreto y volátil. `AddressValidatorService` es volátil (un tercero controla su contrato, su transporte y su disponibilidad) y `new AddressValidatorService()` dentro de `OrderProcessor` es esa mención. No hay abstracción alguna.

- **Arquitectura Limpia: Regla de Dependencia.** Las dependencias del código fuente solo pueden apuntar hacia adentro, hacia las políticas de mayor nivel, y nada en un círculo interior puede conocer ni nombrar algo declarado en uno exterior (Martin, 2017, cap. 22). `OrderProcessor` es un caso de uso; `AddressValidatorService` y la librería HTTP son adaptadores de interfaz y *frameworks*, círculos exteriores. Además, el nivel de una política es su distancia a las entradas y salidas del sistema, y las dependencias deben seguir al nivel, no al flujo de datos (Martin, 2017, cap. 19): que los datos viajen hacia el proveedor no justifica que el negocio dependa del código que habla con él.

- **Arquitectura Hexagonal.** La aplicación solo dialoga con el exterior a través de puertos, con un adaptador por tecnología, para poder ser conducida y probada aislada de los dispositivos y bases de datos reales (Cockburn, 2005). Aquí no hay puerto: el interior habla con un actor externo concreto y en su protocolo. La «conversación» con el proveedor es un detalle tecnológico filtrado hacia dentro, y la aplicación no funciona sin el dispositivo real.

- **Arquitectura Cebolla.** Palermo (2008b) fija cuatro principios: modelo de objetos independiente en el centro; interfaces definidas por las capas internas e implementadas por las externas; acoplamiento hacia el centro; núcleo que se compila y ejecuta separado de la infraestructura. El código actual incumple los tres últimos: el núcleo no define ninguna interfaz, el aco-

plamiento sale del centro y no puede compilarse ni ejecutarse sin `AddressValidatorService` y, transitivamente, la librería HTTP.

**3. Dirección que deben tener las dependencias**

**Toda dependencia del código fuente que cruce la frontera entre el núcleo y el exterior debe apuntar hacia el núcleo.** Aquí:

$$\texttt{HttpAddressValidatorAdapter} \rightarrow \texttt{IAddressValidator} \leftarrow \texttt{OrderProcessor}$$

sin ninguna flecha del núcleo hacia la infraestructura. Lo justifica el principio abierto/cerrado: si un componente A debe protegerse de los cambios de otro B, entonces B debe depender de A (Martin, 2017, cap. 8). El núcleo debe protegerse de los cambios de proveedor, de librería y de formato de datos, así que esos detalles dependen de él. El flujo de control sigue yendo hacia afuera; solo se invierte la dependencia del código fuente en el punto de cruce, mediante polimorfismo (Martin, 2017, caps. 5 y 22).

**4. Consecuencias concretas**

- **Pruebas:** probar `OrderProcessor` exige red o simulaciones a nivel de la librería HTTP, lo que las vuelve lentas, no deterministas y frágiles frente a detalles volátiles (Martin, 2017, cap. 28).
- **Rigidez:** cambiar de proveedor, de librería o de formato de respuesta puede obligar a modificar, recompilar y redistribuir también el núcleo.
- **Contaminación semántica:** las reglas de negocio quedan mezcladas con códigos de estado HTTP.
- **Errores técnicos dentro del negocio:** las excepciones de la librería (red, tiempos de espera) llegan a `OrderProcessor` sin una política de negocio explícita para ellas.

### b) Rediseño de la interacción entre `OrderProcessor` y `AddressValidatorService`

**1. Qué reside en el núcleo y qué fuera**

- **Núcleo (dentro del hexágono):** la interfaz `IAddressValidator` (puerto de salida), `OrderProcessor`, los tipos del dominio que cruzan el puerto (`Address`, `AddressValidationResult`) y el error de dominio `AddressValidationUnavailableException`.
- **Infraestructura (fuera del hexágono):** el adaptador `HttpAddressValidatorAdapter` (el antiguo `AddressValidatorService`, convertido en adaptador), los DTO del proveedor y la configuración de la librería HTTP.
- **Punto de composición (`Main` o `Program.cs`):** el único lugar que conoce ambos lados y enlaza la implementación con la abstracción.

```
NÚCLEO
  OrderProcessor
        |  usa
        v
  IAddressValidator      <-- puerto: lo define y lo posee el núcleo
        ^
========|=========== frontera ===========
        |  implementa
  HttpAddressValidatorAdapter
        |  usa
        v
  Librería HTTP --> Proveedor externo
EXTERIOR
```

Las flechas son dependencias del código fuente: ambas apuntan hacia `IAddressValidator`, es decir, hacia el núcleo. El flujo de control sigue descendiendo hasta el proveedor.

## 2. Código del núcleo

```
// Proyecto Pedidos.Core: sin referencias a HTTP, JSON ni al proveedor
public interface IAddressValidator                          // PUERTO secundario (de salida)
{
    Task<AddressValidationResult> ValidateAsync(Address address);
}

public sealed record AddressValidationResult(bool IsValid, string? Reason = null);

public class AddressValidationUnavailableException : Exception   // error en términos del dominio
{
    public AddressValidationUnavailableException(string message, Exception? inner = null)
        : base(message, inner) { }
}

public class OrderProcessor
{
    private readonly IAddressValidator _addressValidator;      // abstracción, no concreción

    public OrderProcessor(IAddressValidator addressValidator)  // inyección por constructor
        => _addressValidator = addressValidator;

    public async Task ProcessAsync(Order order)
    {
        try
        {
            var result = await _addressValidator.ValidateAsync(order.ShippingAddress);
            if (!result.IsValid) { order.Reject(result.Reason); return; }
        }
        catch (AddressValidationUnavailableException)
        {
            order.MarkPendingAddressReview();   // política de negocio ante la indisponibilidad
            return;
        }
        order.Confirm();                        // ... y el resto de las reglas del pedido
    }
}
```

## 3. Código del adaptador y del punto de composición

```
// Proyecto Pedidos.Infrastructure: referencia a Pedidos.Core, nunca al revés
public class HttpAddressValidatorAdapter : IAddressValidator     // ADAPTADOR secundario
{
    private readonly HttpClient _http;                           // la librería HTTP queda confinada aquí
    public HttpAddressValidatorAdapter(HttpClient http) => _http = http;

    public async Task<AddressValidationResult> ValidateAsync(Address address)
    {
        try
```

```
        {
            var request = ProviderMapper.ToRequest(address);      // dominio -> DTO del proveedor
            var response = await _http.PostAsJsonAsync("v1/validate", request);
            response.EnsureSuccessStatusCode();
            var body = await response.Content.ReadFromJsonAsync<ProviderResponse>();
            return ProviderMapper.ToDomain(body!);                // DTO del proveedor -> dominio
        }
        catch (HttpRequestException ex)                           // fallo técnico -> error de dominio
        {
            throw new AddressValidationUnavailableException("Validador no disponible", ex);
        }
    }
}

// Proyecto Pedidos.Host (Program.cs): único punto que nombra las concreciones
builder.Services.AddHttpClient<IAddressValidator, HttpAddressValidatorAdapter>(
    c => c.BaseAddress = new Uri("https://api.proveedor.com/"));
builder.Services.AddScoped<OrderProcessor>();
```

El punto de composición cumple el papel del componente *Main* (Martin, 2017, cap. 26) y del mecanismo de inyección en tiempo de ejecución que Palermo (2008a) requiere para conectar el núcleo con sus implementaciones; en la guía de De la Torre Llorente et al. (2010) ese mecanismo es un contenedor IoC. Las referencias entre proyectos son `Infrastructure` → `Core` y `Host` → `Core`, `Infrastructure`; `Core` no referencia a ninguno.

## 4. Equivalencia en los tres estilos

| Elemento | Hexagonal | Limpia | Cebolla |
|---|---|---|---|
| `OrderProcessor` | Interior del hexágono | Casos de uso | Servicios de aplicación y dominio (núcleo) |
| `IAddressValidator` | Puerto secundario (de salida) | Interfaz en el círculo de casos de uso, como el puerto de salida del presentador | Interfaz definida por el núcleo |
| `HttpAddress ValidatorAdapter` | Adaptador secundario | Adaptadores de interfaz | Infraestructura |
| Librería HTTP y proveedor | Dispositivo externo | *Frameworks* y *drivers* | Infraestructura (exterior) |
| `Program.cs` | Ensamblaje de puertos y adaptadores | Componente *Main* | Mecanismo de inyección en ejecución |

Tabla 4: Equivalencias según Cockburn (2005), Martin (2017, cap. 22) y Palermo (2008a).

La guía de De la Torre Llorente et al. (2010) aplica el mismo esquema a repositorios y agentes de servicios externos: contratos en las capas internas, implementaciones en la infraestructura y resolución por IoC/DI.

## 5. Criterios de diseño del puerto

- **Lo posee el núcleo.** Se declara en el proyecto del núcleo y según lo que `OrderProcessor` necesita, no según lo que ofrece el proveedor. Para Cockburn (2005), un puerto identifica una conversación con propósito y puede tener muchos adaptadores, según la tecnología.
- **Lenguaje del dominio.** La firma usa `Address` y `AddressValidationResult`; ningún tipo HTTP, JSON o DTO del proveedor aparece en ella. Los datos que cruzan la frontera son estructuras simples, en la forma más cómoda para el círculo interior (Martin, 2017, cap. 22).

- **Errores del dominio.** La indisponibilidad se expresa con una excepción del núcleo. Qué hacer ante ella (rechazar, reintentar luego, pasar a revisión manual) es regla de negocio y vive en `OrderProcessor`; cómo se detecta (tiempos de espera, reintentos) es mecánica del adaptador.
- **Estrecho y nombrado por capacidad.** Una sola operación, la que el caso de uso necesita (Martin, 2017, cap. 10), con un nombre que no menciona tecnología ni proveedor.

#### 6. Justificación: los detalles deben depender de las abstracciones

Antes, el núcleo dependía del detalle. Ahora `OrderProcessor` (alto nivel) solo conoce `IAddressValidator`; `HttpAddressValidatorAdapter` (detalle) depende de la abstracción que implementa; y la abstracción no depende de ningún detalle, porque su firma está libre de HTTP. Se cumplen las dos mitades del DIP. Lo decisivo no es «añadir una interfaz» sino **quién la posee**: si `IAddressValidator` viviera junto al adaptador, en infraestructura, el núcleo tendría que referenciarla y la dependencia seguiría apuntando hacia afuera. Con la interfaz en el núcleo, la dependencia entre proyectos pasa a ser Infraestructura → Núcleo y la Regla de Dependencia se vuelve verificable: el proyecto del núcleo no referencia ni la infraestructura ni paquetes HTTP, algo que puede reforzarse con pruebas de arquitectura automáticas en integración continua.

#### 7. Ruta de refactorización

1. Extraer `IAddressValidator` con lo que `OrderProcessor` realmente usa, expresado en tipos del dominio, y alojarla en el núcleo.
2. Hacer que `AddressValidatorService` implemente la interfaz, renombrarla como adaptador y moverla a infraestructura junto con la librería HTTP y los DTO.
3. Inyectar `IAddressValidator` por constructor en `OrderProcessor` y eliminar el `new`.
4. Registrar la implementación en el punto de composición.
5. Escribir las pruebas del núcleo con un doble de prueba y las del adaptador con un manejador HTTP simulado.

### c) Pruebas aisladas y mantenibilidad

#### 1. Probar `OrderProcessor` sin red ni librería HTTP

`OrderProcessor` solo conoce `IAddressValidator`, así que en las pruebas se le entrega un doble de prueba en lugar del adaptador. Es lo que busca Cockburn (2005), una aplicación conducida por pruebas automatizadas sin dispositivos reales; lo que exige Palermo (2008b) al pedir que el núcleo se ejecute sin infraestructura; y lo que Martin (2017, cap. 28) describe al ubicar las pruebas como el círculo más externo, que depende hacia adentro. La guía de De la Torre Llorente et al. (2010) plantea la inyección de dependencias en esa misma línea, para poder sustituir implementaciones reales por simuladas.

```
public class FakeAddressValidator : IAddressValidator           // doble de prueba programable
{
    public AddressValidationResult Response { get; set; } = new(true);
    public bool Unavailable { get; set; }

    public Task<AddressValidationResult> ValidateAsync(Address address) =>
        Unavailable
            ? Task.FromException<AddressValidationResult>(
                  new AddressValidationUnavailableException("simulado"))
            : Task.FromResult(Response);
```

```
}

public class OrderProcessorTests          // sin red, sin HTTP, sin la librería HTTP
{
    [Fact]
    public async Task Rechaza_el_pedido_si_la_direccion_es_invalida()
    {
        var validator = new FakeAddressValidator { Response = new(false, "No existe") };
        var order = new OrderBuilder().Build();                  // ayudante de pruebas

        await new OrderProcessor(validator).ProcessAsync(order);

        Assert.Equal(OrderStatus.Rejected, order.Status);
    }

    [Fact]
    public async Task Queda_en_revision_si_el_validador_no_esta_disponible()
    {
        var validator = new FakeAddressValidator { Unavailable = true };
        var order = new OrderBuilder().Build();

        await new OrderProcessor(validator).ProcessAsync(order);

        Assert.Equal(OrderStatus.PendingAddressReview, order.Status);
    }
}
```

**Qué se gana:**

- **Aislamiento y determinismo:** el resultado depende solo de la lógica de negocio, no del proveedor ni de la red; sin entrada/salida, las pruebas son repetibles y rápidas.
- **Escenarios difíciles:** se simulan sin esfuerzo la dirección inválida, el tiempo de espera o el proveedor caído, casos raros o costosos de provocar con el servicio real.
- **Independencia de la librería:** el proyecto de pruebas del núcleo no referencia la librería HTTP; actualizarla o sustituirla no lo afecta, porque depende solo del contrato abstracto.

### 2. Pruebas del adaptador

HTTP no desaparece de la estrategia de pruebas: queda confinado a un componente delgado. El adaptador se verifica con pocas pruebas de integración (traducción de datos en ambos sentidos y conversión de fallos técnicos en `AddressValidationUnavailableException`), con un manejador HTTP simulado o el entorno de pruebas del proveedor. Un doble de prueba nunca valida al proveedor real, por eso ambos tipos de pruebas se complementan.

### 3. Mantenibilidad y cambio del proveedor

Cambiar de proveedor significa escribir otro adaptador que implemente `IAddressValidator` (por ejemplo, `OtroProveedorAddressValidatorAdapter`) y cambiar una línea del punto de composición. `OrderProcessor` y sus pruebas no se tocan: el núcleo queda cerrado a la modificación y abierto a la extensión (Martin, 2017, cap. 8). Además, un puerto admite varios adaptadores a la vez (Cockburn, 2005): el HTTP de producción, uno en memoria para desarrollo, un segundo proveedor de respaldo o un adaptador que envuelve a otro para añadir caché, todo sin modificar el negocio.

| Cambio | Antes | Después |
|---|---|---|
| Cambiar de proveedor | `AddressValidatorService` y, según cuánto se haya filtrado su semántica, `OrderProcessor` y sus pruebas | Un adaptador nuevo y una línea en el punto de composición |
| Actualizar o sustituir la librería HTTP | La clase concreta; puede afectar al núcleo por la dependencia transitiva | Solo el adaptador |
| El proveedor cambia su formato de datos | La clase concreta y todo lo que use su semántica | Solo el mapeo del adaptador |
| Pasar de HTTP a otro protocolo | Reescritura con riesgo para el núcleo | Otro adaptador del mismo puerto |
| Cambiar la política ante indisponibilidad | Entrelazada con código HTTP | Una decisión de negocio localizada en `OrderProcessor` |

Tabla 5: Impacto de los cambios habituales antes y después del rediseño.

### 4. Costos y precauciones

- **Costo:** una interfaz, una clase adaptadora, mapeo y configuración de inyección: más código e indirección. Es un *trade-off*, como toda decisión arquitectónica (Richards & Ford, 2020, cap. 1); se justifica ante dependencias volátiles y externas como esta, no ante clases estables (Martin, 2017, cap. 11).
- **Abstracción con fugas:** si el puerto expone tipos HTTP o del proveedor, el aislamiento es solo aparente.
- **Interfaz mal ubicada:** declarada junto a su implementación, no invierte nada.
- **Dobles que mienten:** un doble no equivale al proveedor real; por eso se mantienen las pruebas del adaptador.
- **Dependencias ocultas:** conviene preferir la inyección por constructor a `new` o a localizadores de servicios; solo el punto de composición nombra clases concretas (Martin, 2017, cap. 26).

## Conclusión

La implementación original no falla por usar un servicio externo, sino por la dirección de su dependencia: el negocio depende de un detalle. Invertirla (puerto en el núcleo, adaptador en la infraestructura y composición en el borde) hace coincidir a las arquitecturas Hexagonal, Limpia y Cebolla en lo esencial: un núcleo independiente de la tecnología, detalles intercambiables y pruebas rápidas y aisladas. El precio es una indirección adicional, que vale la pena en fronteras volátiles como esta.

# 5. Análisis de un problema real: el *Kitty Problem* y el Principio de Abierto/Cerrado

*Responsable: Cristian Miguel Castañeda*

## Descripción del problema

El *Kitty Problem* se desarrolla en el capítulo de *Clean Architecture* dedicado a los servicios (Martin, 2017, cap. 27).[1] Parte de un sistema agregador de taxis que conoce a muchos proveedores de una ciudad y permite a los clientes pedir viajes según criterios como la hora de recogida, el costo o el nivel de lujo. El sistema está descompuesto en servicios según su función: `TaxiUI` atiende a los clientes desde sus dispositivos móviles; `TaxiFinder` examina los inventarios de los proveedores y determina qué taxis son candidatos; `TaxiSelector` elige entre ellos el que mejor se ajusta a los criterios del cliente; y `TaxiDispatcher` encarga el viaje al taxi elegido.

Un día la empresa decide ofrecer un servicio nuevo: la entrega de gatitos a domicilio o en el lugar de trabajo, transportados por los taxis del sistema. El requisito arrastra reglas propias: no todos los proveedores participan en el programa, y hay conductores —y clientes— alérgicos a los gatos que deben quedar fuera de ciertas asignaciones. Al revisar el diagrama de servicios, la respuesta a cuántos de ellos deben cambiar es: todos. La interfaz debe permitir pedir gatitos, la búsqueda debe limitarse a los proveedores participantes, la selección debe respetar las nuevas restricciones y el despacho debe gestionar un tipo de encargo distinto. Además, esos cambios deben desarrollarse y desplegarse de forma coordinada.

Martin (2017, cap. 27) usa el ejemplo para desmontar dos supuestos beneficios de los servicios, a los que llama falacias: que estén desacoplados y que puedan desarrollarse y desplegarse de forma independiente. Los servicios estaban separados físicamente, pero quedaron acoplados por un requisito transversal. Su diagnóstico es que las descomposiciones funcionales son muy vulnerables a las funcionalidades nuevas que atraviesan todos los comportamientos existentes, y que las fronteras arquitectónicas no caen entre los servicios, sino que los atraviesan y los dividen en componentes.

## ¿Por qué la solución de dividir el sistema en servicios basados en verbos o entidades funcionales (como `TaxiFinder` o `TaxiSelector`) llevó a este problema?

El diseño por verbos falló porque `TaxiFinder` y `TaxiSelector` representan acciones genéricas, no áreas de negocio. Al no existir un dominio donde el concepto *gatito* viviera de forma natural, el nuevo requisito se filtró a todos los servicios, que compartían una noción implícita de viaje como transporte de personas. Añadir gatitos obligó así a modificar `TaxiFinder`, `TaxiSelector` y `TaxiDispatcher` simultáneamente, violando el *Bounded Context* de DDD (Evans, 2003), ya que cada servicio acabó conociendo todos los tipos de petición, y el Principio de Abierto/Cerrado (Martin, 2017, cap. 8), ya que extender el sistema requirió modificar código existente en lugar de añadir código nuevo.

## Basándose en los principios de DDD y la Arquitectura Onion/Clean, ¿cómo se podría haber diseñado el sistema para que un cambio de este tipo afectara a un solo componente o servicio?

Aplicando DDD (Evans, 2003), el sistema se habría organizado en Contextos Delimitados por dominio de negocio como `Rides` y `Kittens`, de modo que el concepto *gatito* viviera íntegramente dentro de su propio contexto y no se filtrara a los servicios existentes. Aplicando la

[1] El enunciado del seminario lo atribuye a Richards y Ford (2020); el ejemplo, con los servicios `TaxiUI`, `TaxiFinder` y `TaxiSelector` y los componentes `Rides` y `Kittens`, se expone en Martin (2017, cap. 27), que es la fuente que se sigue en esta respuesta.

Arquitectura Onion/Clean (Palermo, 2008a; Martin, 2017), el dominio `Kittens` se habría aislado en el centro, dependiendo solo de un puerto (`TransportProvider`) e ignorando por completo la infraestructura. Un adaptador (`TaxiTransportAdapter`), en el sentido de Cockburn (2005), habría traducido las peticiones de `Kittens` al lenguaje genérico de `Rides`, siendo el único componente que conoce ambos mundos. Finalmente, `Kittens` se habría registrado como *plugin* en el arranque del sistema, de forma que añadir el nuevo requisito consistiera en añadir archivos nuevos y una llamada de registro, sin modificar `TaxiFinder`, `TaxiSelector` ni `TaxiDispatcher`. El resultado es que el cambio afecta a un solo componente (el dominio `Kittens` con su adaptador), cumple el Principio de Abierto/Cerrado (extensión sin modificación) y respeta la dirección de dependencias de Onion (el dominio no depende de la infraestructura).

**Proponga una solución utilizando componentes como `Kittens` y `Rides` que se añadan como plugins, explicando cómo esto respeta el Principio de Abierto/-Cerrado (OCP) y qué papel juegan los Adaptadores en esta extensión.**

La solución sería diseñar el sistema como un núcleo estable (`Rides`) más *plugins* enchufables (`Kittens`). El núcleo `Rides` contiene la lógica genérica de despacho de viajes y expone un método genérico (`dispatchGeneric`) capaz de procesar cualquier tipo de carga, tanto personas como gatitos, sin conocer su naturaleza concreta. Además, el sistema cuenta con un registro de *plugins* donde cada módulo nuevo se inscribe en el arranque. El *plugin* `Kittens` es un módulo independiente que contiene su propio dominio (`Kitten`), su caso de uso (`DeliverKittenUseCase`), un puerto (`TransportProvider`) que declara qué necesita del exterior, y un adaptador (`TaxiTransportAdapter`) que implementa ese puerto traduciendo las peticiones al lenguaje de `Rides`.

Para añadir el *plugin*, se crea una función de registro (`register_kittens_plugin`) que instancia el adaptador, instancia el caso de uso y registra el *endpoint* `kittens/deliver` en el registro de *plugins*. En el arranque de la aplicación se registra el sistema base `Rides` (que ya existía) y, a continuación, se llama a `register_kittens_plugin`. El resultado es que añadir gatitos consiste en añadir archivos nuevos y una llamada de registro, sin modificar ni una línea de `TaxiFinder`, `TaxiSelector`, `TaxiDispatcher` ni `Ride`.

Esto respeta el Principio de Abierto/Cerrado (Martin, 2017, cap. 8): el sistema queda abierto a la extensión (nuevos *plugins*) y cerrado a la modificación (el núcleo no se toca). Los adaptadores son la pieza clave (Cockburn, 2005): traducen entre el lenguaje de `Kittens` y el de `Rides`, son el único punto de acoplamiento y permiten que ambos dominios evolucionen por separado sin conocerse mutuamente.

Esta propuesta coincide en lo esencial con la solución que plantea el propio Martin (2017, cap. 27): conservar la lógica general en clases base abstractas y llevar lo específico de los viajes a un componente `Rides` y lo de los gatitos a un componente `Kittens`, que sobrescriben esas clases mediante patrones como *Template Method* o *Strategy* y respetan la Regla de Dependencia. Así, al incorporar la nueva funcionalidad solo cambia la interfaz de usuario; el resto se añade como un componente nuevo que se carga en tiempo de ejecución. Martin concluye que los servicios no tienen por qué ser pequeños monolitos: pueden diseñarse con los principios SOLID y con una estructura interna de componentes que siga la Regla de Dependencia, de modo que añadir funcionalidades cumpla el Principio de Abierto/Cerrado.

# 6. Evaluación de la Arquitectura N-Capas frente a requisitos de atributos de calidad

*Responsable: Lázaro Javier Aragon*

## Impacto de las capas abiertas en el aislamiento de capas y la mantenibilidad a largo plazo

El **Aislamiento de Capas** es el principio por el cual, en una arquitectura N-Capas, cada capa está encapsulada y separada lógicamente de las demás; es decir, una capa solo debe comunicarse con la capa inmediatamente inferior a través de interfaces bien definidas, sin saltarse capas ni acceder directamente a los detalles internos de otra. A este tipo de capas se les denomina **capas cerradas** y permiten preservar la separación de responsabilidades, de modo que un cambio en una capa no debería afectar a las demás. Por otro lado, mientras que las **capas cerradas** obligan a una petición a pasar por cada capa, una **capa abierta** permite que las peticiones se salten capas intermedias. Técnicas como el patrón *fast-lane reader* aprovechan la estructura de **capas abiertas** para permitir que la capa de presentación acceda directamente a la capa de persistencia, evitando la lógica de negocio innecesaria y mejorando el rendimiento. Pero la introducción de **capas abiertas** rompe con el aislamiento de las capas, dando como resultado que un cambio realizado en una capa obligue a modificar al resto de las capas conectadas a ella. Además, se pueden saltar reglas de negocio, validaciones, seguridad o auditoría que residían en las capas intermedias, lo que hace el sistema más frágil y difícil de probar y evolucionar.

En síntesis, las capas abiertas son una optimización táctica de rendimiento que degrada el aislamiento y, con él, la mantenibilidad, por lo que no constituyen una solución estructural a los problemas de la arquitectura N-Capas.

## El anti-patrón *sinkhole* y la regla 80/20

El **anti-patrón *sinkhole*** ocurre cuando muchas peticiones atraviesan las capas **sin que estas ejecuten lógica de negocio real**, limitándose a pasar los datos a la siguiente capa. Richards y Ford lo definen como el escenario en el que «las capas solo pasan datos sin agregar valor» (Richards & Ford, 2020). Cada capa añade latencia y complejidad sin beneficio funcional.

Para diagnosticar si este anti-patrón está presente, Richards (2015) propone la **regla 80/20** como heurística práctica:

- Si $\leq 20\,\%$ de las peticiones son *sinkholes*, el nivel es aceptable.
- Si $\approx 80\,\%$ de las peticiones son *sinkholes*, se ha elegido la arquitectura equivocada.

La relación con las capas abiertas es directa: **abrir capas puede aliviar el síntoma** (el rendimiento), pero **no corrige la causa raíz**. Si la mayoría de las peticiones no necesitan lógica de negocio, el problema no es que las capas estén cerradas, sino que **la partición técnica de la arquitectura N-Capas no se alinea con el dominio del negocio** (Richards, 2015). Abrir capas solo enmascara esta desalineación estructural a costa de sacrificar el aislamiento y la mantenibilidad.

## Por qué Hexagonal y Limpia son inherentemente más adecuadas para la escalabilidad y la agilidad

La arquitectura N-Capas se particiona por **rol técnico** (presentación, negocio, datos). Esto implica que un cambio en un caso de uso de negocio obliga a tocar varias capas, y que la **unidad de despliegue es única**, lo que limita la escalabilidad y la agilidad. Por el contrario, las arquitecturas Hexagonal y Limpia, especialmente cuando se combinan con **Domain-Driven Design**

**(DDD)**, permiten organizar el código en torno a **contextos delimitados** (*Bounded Contexts*) y **agregados**. DDD ayuda a «dividir el dominio complejo en múltiples contextos delimitados» y a «modelar cada contexto delimitado por separado» (Evans, 2003), lo que **modulariza el sistema por áreas de negocio** y facilita la escalabilidad y la evolución independiente de cada módulo.

Además, en la arquitectura N-Capas, aunque exista una interfaz, la dirección de dependencia es descendente: la lógica de negocio sigue dependiendo de la capa de datos. En la **arquitectura Hexagonal**, el núcleo de la aplicación (la lógica de negocio) está **aislado de los dispositivos de tiempo de ejecución y las bases de datos**, y se puede desarrollar y probar de forma independiente (Cockburn, 2005). La **arquitectura Limpia** lleva este principio a su formulación más rigurosa con la **Regla de Dependencia**: las dependencias del código fuente solo pueden apuntar hacia adentro, y las capas externas (UI, base de datos, frameworks) son «plugins» del núcleo de negocio (Martin, 2017). Esto hace que el núcleo sea **agnóstico a la tecnología** y permite cambiar la base de datos, la UI o un servicio externo **sin tocar la lógica de negocio**, lo cual es la base de la **agilidad tecnológica** a largo plazo.

Al aislar el núcleo, las **pruebas unitarias de la lógica de negocio se pueden ejecutar sin necesidad de base de datos, UI o servicios externos**. La arquitectura Hexagonal «tiene como objetivo crear arquitecturas débilmente acopladas donde los componentes de la aplicación puedan probarse de forma independiente, sin dependencias de almacenes de datos ni interfaces de usuario» (Cockburn, 2005). Esta **alta testabilidad** acelera los ciclos de retroalimentación y permite evolucionar el sistema con seguridad, algo que en una arquitectura N-Capas con capas abiertas se vuelve más difícil debido al acoplamiento introducido.

## Conclusión

Las optimizaciones como las **capas abiertas** y el *fast-lane reader* son **parches tácticos** que mitigan un síntoma (el rendimiento) a costa de un atributo de calidad fundamental (el aislamiento y, por tanto, la mantenibilidad). **No abordan la causa raíz**. Para requisitos de **alta escalabilidad y agilidad**, las arquitecturas **Hexagonal** y **Limpia** ofrecen una solución **estructural**, al hacer que el núcleo de negocio sea independiente, testeable y verdaderamente desacoplado de los detalles tecnológicos.

## Referencias


Cockburn, A. (2005, 4 de septiembre). *The hexagonal (Ports & Adapters) architecture.* https://alistair.cockburn.us/hexagonal-architecture/

De la Torre Llorente, C., Zorrilla Castro, U., Ramos Barroso, M. A., & Calvarro Nelson, J. (2010). *Guía de arquitectura N-Capas orientada al dominio con .NET 4.0* (Borrador). Microsoft Ibérica.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software.* Addison-Wesley.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design.* Prentice Hall.

Palermo, J. (2008a, 29 de julio). The onion architecture: Part 1. *Programming with Palermo.* https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/

Palermo, J. (2008b, 4 de agosto). The onion architecture: Part 3. *Programming with Palermo.* https://jeffreypalermo.com/2008/08/the-onion-architecture-part-3/

Richards, M. (2015). *Software architecture patterns.* O'Reilly Media.

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach.* O'Reilly Media.


## Declaración de Uso de la Inteligencia Artificial

Para la elaboración de este seminario, el equipo utilizó herramientas de inteligencia artificial generativa: **Claude** (Anthropic, modelo Claude Opus 5.5) y **ChatGPT** (OpenAI, GPT-4). Se emplearon con los siguientes propósitos:

- **Generación de código LaTeX y maquetación de documentos:** conversión de las respuestas individuales (elaboradas en Word, texto plano y LaTeX) a un único documento LaTeX, diseño de la portada, tablas, diagramas en TikZ y listados de código, unificación del formato de citas y de la bibliografía según las normas APA, comprobación de las citas de fuentes web frente a los textos originales y compilación a PDF.
- **Generación de ideas para la realización de los documentos:** propuestas de estructura para las respuestas y de ejemplos y diagramas que ilustran los conceptos analizados.
- **Redacción de borradores:** borrador de la respuesta a la Pregunta 2 y de los fragmentos añadidos a la Pregunta 5 (la descripción del *Kitty Problem* y su relación con la solución que propone Martin).
- **Resolución de dudas sobre el contenido de la asignatura:** aclaración de conceptos como la Regla de Dependencia, los puertos y adaptadores, los contextos delimitados, el Principio de Abierto/Cerrado o el anti-patrón *sinkhole*, contrastando siempre las respuestas con la bibliografía orientada para el seminario.

Los resultados obtenidos se utilizaron como apoyo y punto de partida: todo el contenido generado fue revisado, verificado frente a las fuentes citadas y adaptado por los integrantes del equipo, quienes asumen la responsabilidad final del documento.